

Aulas 14 a 17

Otimizações RTL independente de tecnologia

Autores

Gabriel A. F. Souza, Gustavo D. Colletta, Leonardo B. Zoccal, Odilon O. Dutra

Unifei

Histórico de Revisões

2 de dezembro de 2025	1.0	Primeira versão do documento.
-----------------------	-----	-------------------------------

Tópicos

- Introdução
- Otimizações RTL Independente de Tecnologia
- Aplicação Prática
- Atividades Hands-on

Introdução

Objetivos

- Entender como a ferramenta Cadence Genus traduz o código RTL para um netlist de portas lógicas.
- Entender os conceitos fundamentais de SDC (Synopsys Design Constraints), Setup e Hold Time na Análise de Temporização Estática (STA).
- Otimizações RTL: Focar em técnicas de projeto em nível de código (independentes da tecnologia de célula padrão) que são cruciais para atingir as restrições de frequência (clock), com ênfase no Pipelining (Segmentação).
- Aplicação Prática: Como modificar um código Verilog para incluir pipelining e resolver uma violação de Setup Time.

Otimizações RTL Independente de Tecnologia

Otimizações RTL

Estas técnicas alteram a estrutura do código RTL para criar caminhos combinacionais mais curtos, antes que o Genus comece a mapear para a tecnologia alvo.

Retiming

Movimenta registradores através da lógica combinacional, sem alterar a função do circuito (a latência permanece a mesma, mas a distribuição do atraso de clock muda).

Objetivo: Balancear os atrasos nos caminhos combinacionais para que todos sejam o mais próximos possível do período de clock desejado. O Genus tem recursos para realizar retiming automático.

Pipelining

Divide um caminho combinacional longo em vários segmentos menores, inserindo registradores (flip-flops) intermediários.

Vantagem: Reduz o atraso máximo de cada segmento, permitindo um período de clock mais curto e, conseqüentemente, uma frequência de operação mais alta.

Desvantagem: Aumenta a latência (o tempo total que um dado leva para atravessar o circuito do início ao fim) e a área (devido aos registradores extras)

Aplicação Prática

Multiplicador

Considere o circuito Multiplicador utilizando a estrutura com Carry Save Adder apresentada no módulo SD122.

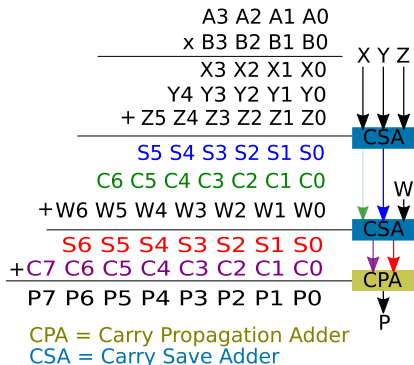


Figura 1: Estrutura do multiplicador

Multiplicador

Considere a seguinte implementação do multiplicador com entradas e saída registradas.

```
1 // Módulo Multiplicador CSA 4x4 com Registros
2 module multiplier_csa (
3     input clk,
4     input reset,
5     input [3:0] multiplicand_in,
6     input [3:0] multiplier_in,
7     output [7:0] product_out
8 );
9
10 // --- 1. Registros de Entrada ---
11 reg [3:0] r_multiplicand;
12 reg [3:0] r_multiplier;
13
14 always @(posedge clk or posedge reset) begin
15     if (reset) begin
16         r_multiplicand <= 4'h0;
17         r_multiplier <= 4'h0;
18     end else begin
19         r_multiplicand <= multiplicand_in;
20         r_multiplier <= multiplier_in;
21     end
22 end
```

Multiplicador

```
24 // --- 2. Lógica Combinacional (Core do Multiplicador 4x4) ---
25 // Produtos Parciais (PP) - 4 produtos, 8 bits cada.
26 wire [7:0] partial_products [3:0];
27
28 // Geração dos 4 Produtos Parciais (PP0 a PP3)
29 assign partial_products[0] = r_multiplier[0] ? (r_multiplicand << 0) : 8'h0; //
   PP0
30 assign partial_products[1] = r_multiplier[1] ? (r_multiplicand << 1) : 8'h0; //
   PP1
31 assign partial_products[2] = r_multiplier[2] ? (r_multiplicand << 2) : 8'h0; //
   PP2
32 assign partial_products[3] = r_multiplier[3] ? (r_multiplicand << 3) : 8'h0; //
   PP3
33
34 // Sinais Intermediários de Somas (Sum) e Carrys (Carry)
35 wire [7:0] sum_stage1, sum_stage2;
36 wire [7:0] carry_stage1, carry_stage2;
37 wire [7:0] combinational_product;
38
39 // CSA 1: Reduz 3 PPs (PP0, PP1, PP2) em 2 vetores (Sum1, Carry1)
40 // Usamos PP2 como A, PP0 como B e PP1 como Cin
41 parameterized_csa #(8) csa_stage1 (
42     .A(partial_products[2]),
43     .B(partial_products[0]),
44     .Cin(partial_products[1]),
45     .Sum(sum_stage1),
46     .Cout(carry_stage1)
47 );
```

Multiplicador

```
49 // CSA 2: Reduz o último PP (PP3) e os vetores intermediários (Sum1, Carry1)
50 // PP3 é o último produto parcial a ser somado
51 parameterized_csa #(8) csa_stage2 (
52     .A(partial_products[3]),
53     .B(sum_stage1),
54     .Cin(carry_stage1 << 1), // 0 vetor Carry deve ser deslocado em 1 bit
55     .Sum(sum_stage2),
56     .Cout(carry_stage2)
57 );
58
59 // Soma Final (RCA): Reduz os 2 vetores finais (Sum2, Carry2) no produto de 8
60 // bits
61 assign combinational_product = sum_stage2 + (carry_stage2 << 1);
```

Multiplicador

```
63 // --- 3. Registro de Saída ---
64 reg [7:0] r_product;
65
66 always @(posedge clk or posedge reset) begin
67     if (reset)
68         r_product <= 8'h0;
69     else
70         r_product <= combinational_product;
71 end
72
73 assign product_out = r_product;
74 endmodule
```


Multiplicador

```
1 // Modulo CSA parametrizavel
2 module parameterized_csa #(parameter WIDTH = 4) (
3     input  [WIDTH-1:0] A,
4     input  [WIDTH-1:0] B,
5     input  [WIDTH-1:0] Cin,
6     output [WIDTH-1:0] Sum,
7     output [WIDTH-1:0] Cout
8 );
9     genvar i;
10    generate
11        for (i = 0; i < WIDTH; i = i + 1) begin : CSA_BITS
12            assign Sum[i] = A[i] ^ B[i] ^ Cin[i];
13            assign Cout[i] = (A[i] & B[i]) | (B[i] & Cin[i]) |
14                               (A[i] & Cin[i]);
15        end
16    endgenerate
17 endmodule
```

Constraints

Descrição do Constraint	Valor / Parâmetro
Período de Clock (CLK)	1 ns
Tempo de Transição (Subida) do Clock	0.2 ns
Tempo de Transição (Descida) do Clock	0.3 ns
Incerteza do Clock (Jitter/Skew)	0.1 ns
Atraso das Entradas	0.1 ns
Atraso de Saída	0.1 ns

Atividades Hands-on

Atividade

- Realizar a síntese do circuito multiplicador apresentado e analisar os resultados obtidos.
- Realizar otimizações em nível RTL reorganizando o circuito com Pipelining conforme necessário e executar o fluxo de síntese até as constraints de temporização serem atingidas.
- Valide as modificações realizadas no circuito por meio de um testbench.
- Documentar as etapas realizadas e analisar/comentar os resultados obtidos (em termos de temporização, área e potência). Entregar documento em arquivo PDF.